The Journal of Open Source Software

A Graphical User Interface for Well-Connected Clusters

Joao Alfredo Cardoso Lamy [1], **Tomas Alessi** [1], **Tandy Warnow** [2], **George Chacko** [2], **and Minhyuk Park** [2¶]

1 Insper Instituto de Ensino e Pesquisa, Sao Paulo, Brazil **2** Siebel School of Computing and Data Science, University of Illinois Urbana-Champaign, IL 61801, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 05 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Introduction

Community detection in networks has broad applications (Fortunato & Newman, 2022). Beyond the intuitive expectation that communities have greater edge density relative to network background, an important, but sometimes overlooked, quality of "good clusters" is that they should be internally well-connected (Park, Tabatabaee, et al., 2024; Traag et al., 2019). Post-clustering techniques such as the Connectivity Modifier (CM) (Park, Tabatabaee, et al., 2024; Ramavarapu et al., 2024) and Well-Connected Clusters (WCC) (Park, Feng, et al., 2024) modify a clustering to ensure that all clusters meet a minimum standard defined by their mincut. Here, we present a simple user-friendly graphical user interface (GUI) that reduces the burden of installation and the complexity of command line operations for non-expert users. The GUI enables a user to cluster a network and perform a post-treatment to enforce well-connectedness of clusters.

Background

The CM pipeline

CM was designed to enforce well-connectedness in clusters generated during a community detection process (Park, Tabatabaee, et al., 2024; Ramavarapu et al., 2024). The basis by which a cluster is considered well-connected is defined by the size of its min-cut, the smallest set of edges that need to be removed in order for the cluster to be split. In the CM pipeline, if the min-cut of a cluster is above a user-specified threshold, a cluster is considered well-connected. If not, then the min-cut is removed and the products of the cut (two clusters) are re-clustered and re-tested for their min-cuts until every community is well-connected. The threshold specified in (Park, Tabatabaee, et al., 2024) is a mild standard of $log_{10}n$, with n being the number of nodes in the cluster but the pipeline allows users to specify their own criteria through custom functions.

Well-Connected Clusters

One use of the WCC or CM post-treatment is to address the "resolution limit" (Fortunato & Barthelemy, 2007), which is where a clustering method is unable to find the valid communities that are too small for the method. The classical example is a ring of cliques, where each pair of adjacent cliques are connected by a single edge. As established in (Fortunato & Barthelemy, 2007), for large enough numbers of cliques, modularity clustering will group adjacent cliques into single clusters, which is a failure to detect the obvious communities. Here we show that the same problem occurs for Leiden-CPM, given a ring of 40 cliques, each of size 6. Before CM or WCC, Leiden-CPM will group 5 of the cliques together into a single cluster, but following with CM or WCC post-treatment breaks up these incorrect clusters so that only the cliques are returned as clusters (Figure 1).

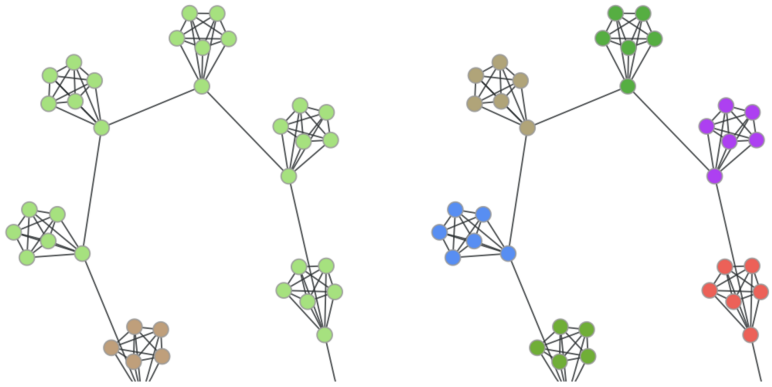

Figure 1: Leiden-CPM(0.01) without and with WCC treatment on a ring-of-cliques network (40 cliques with 6 nodes each) Left: Zoomed in view of Leiden-CPM with resolution value 0.01. Right: Zoomed in view of Leiden-CPM with resolution value 0.01 and post-treated using WCC. The visualization uses colors to denote different clusters. Leiden-CPM by itself merges adjacent cliques into a single large cluster whereas WCC post-treatment is able to separate out individual cliques into their own clusters.

Statement of need

Clustering has broad applications. The selection of a clustering method and choice of parameter settings is often assisted by exploratory analysis. A user-friendly GUI enables such initial exploratory analysis and lowers the barrier for entry. The GUI described here can also be used as an instructional tool in introductory classes on community detection.

The GUI

GUI Architecture

The GUI is modularized into front-end and back-end components. The GUI is implemented in Python leveraging Streamlit (Streamlit Inc., 2019) for the front-end and FastAPI (Ramírez, 2018) for the back-end and supports **remote hosting** on a website. We show in Figure 2 the main interface for the GUI.

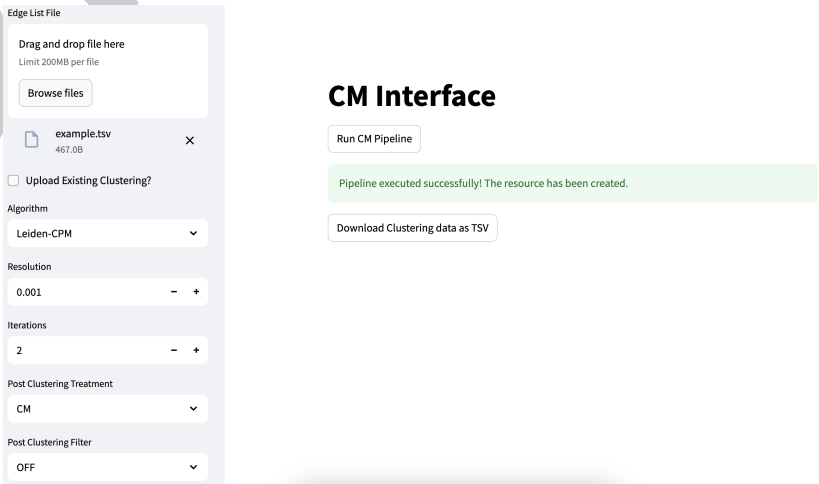

Figure 2: Main interface Example successful execution of clustering using the Leiden algorithm optimizing for CPM and following up with CM post-treatment.

DRAFT

In Figure 3, we show the different clustering options available in the GUI. At present, support is provided for 4 different clustering algorithms: Leiden-CPM (Constant Potts Model) (Traag et al., 2011, 2019), Leiden-Modularity (Traag et al., 2019), Infomap (Rosvall & Bergstrom, 2008), and Stochastic Block Models (SBM) (Peixoto, 2014). Each algorithm takes a set of parameters that must be specified before running the pipeline. The parameters are explained in the CM pipeline documentation and in the CM-GUI documentation.

The user is required to upload a network as an edge list, which is then clustered using the method selected by the user, after which well-connectedness is enforced. The user may also upload a pre-computed clustering of the network and skip the initial clustering stage of the CM pipeline. Results can be downloaded at the end of the run by clicking the "Download Clustering data as CSV" button.

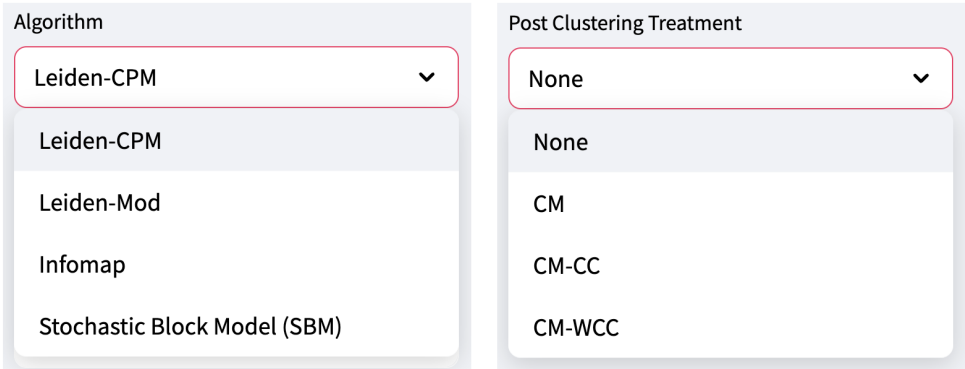

Figure 3: Example options for GUI Left: Choices for clustering algorithms. Right: Choices for post-treatment. In the GUI, the algorithm choices dropdown menu specifies the clustering algorithm for the initial clustering. The post-treatment specifies which, if any, post-treatment should be applied to the clustering. If an existing clustering is not uploaded, the algorithm must be specified to use CM as post-treatment.

Installing the GUI

To install the CM pipeline GUI, the user has two options: via Docker or a manual install.

We recommend the Docker install as it simplifies installation of necessary packages for running the GUI. A local installation option without Docker is available and described in the README of the repository.

Conclusions

The GUI makes the CM pipeline accessible to entry-level users and can be used as an instructional tool. The size of the networks that can be handled by this GUI depend on the hardware that it is installed on. Future work planned will include retrieving basic cluster statistics and visualizations through the GUI on the clusterings produced.

Acknowledgements

We would like to thank Ian Wei Chen and The-Anh Vu-Le for their help in testing the software. Work on this project was supported by funds from the Illinois-Insper Partnership.

References

Fortunato, S., & Barthelemy, M. (2007). Resolution limit in community detection. *Proceedings of the National Academy of Sciences*, *104*(1), 36–41. https://doi.org/10.1073/pnas.0605965104

Fortunato, S., & Newman, M. E. J. (2022). 20 years of network community detection. *Nature Physics*, *18*, 848–850. https://doi.org/10.1038/s41567-022-01716-7

Park, M., Feng, D. W., Digra, S., Vu-Le, T.-A., Chacko, G., & Warnow, T. (2024). Improved Community Detection using Stochastic Block Models. *International Conference on Complex Networks and Their Applications*, 103–114. https://doi.org/10.1007/978-3-031-82435-7_9

Park, M., Tabatabaee, Y., Ramavarapu, V., Liu, B., Pailodi, V. K., Ramachandran, R., Korobskiy, D., Ayres, F., Chacko, G., & Warnow, T. (2024). Well-connectedness and community detection. *PLOS Complex Systems*, *1*(3), e0000009. https://doi.org/10.1371/journal.pcsy.0000009

Peixoto, T. P. (2014). The graph-tool Python library. *Figshare*. https://doi.org/10.6084/m9.figshare.1164194

Ramavarapu, V., Ayres, F. J., Park, M., Pailodi, V. K., Lamy, J. A. C., Warnow, T., & Chacko, G. (2024). CM++ - A Meta-method for Well-Connected Community Detection. *Journal of Open Source Software*, *9*(93), 6073. https://doi.org/10.21105/joss.06073

Ramírez, S. (2018). *FastAPI*. https://fastapi.tiangolo.com/

Rosvall, M., & Bergstrom, C. T. (2008). Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences*, *105*(4), 1118–1123. https://doi.org/10.1073/pnas.0706851105

Streamlit Inc. (2019). *Streamlit*. https://streamlit.io/

Traag, V. A., Van Dooren, P., & Nesterov, Y. (2011). Narrow scope for resolution-limit-free community detection. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, *84*(1), 016114. https://doi.org/10.1103/PhysRevE.84.016114

Traag, V. A., Waltman, L., & Van Eck, N. J. (2019). From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, *9*(1), 1–12. https://doi.org/10.1038/s41598-019-41695-z

DRAFT